Interface: runner.py <-> Agent Class

1. Constructor

Signature	Description
__init__(player: str)	Minimal variant without path.
__init__(player: str, agent_path: str)	Extended variant. runner.py detects agent_path via inspect.signature and passes the load path from the CSV. If agent_path is not None, the constructor MUST call self.load_agent(agent_path) internally.

2. Required Attributes (Instance Variables)

Attribute	Type	Description
agent.player	str	Must be "player_0" or "player_1" - used as key for obs, reward, actions and dones.
agent.name	str	Display name in the replay viewer (required in eval mode).

3. Methods

Method	Required	Signature	Description
act	[x] Yes	act(step: int, obs: dict) -> np.ndarray	Core method - returns array of shape (max_units, 3). Called every step.
load_agent	[x] If agent_path	load_agent(load_path: str)	Loads saved agent state from path. Called by the constructor. If file does not exist, agent starts untrained.
update	[ ] Optional	update(step, obs, reward, action, terminated, next_obs)	Learning step after each transition. Called in training mode only.
set_env_cfg	[ ] Optional	set_env_cfg(env_cfg: dict)	Passes environment parameters (map size, max steps etc.) at the start of each game.
save_agent	[ ] Optional	save_agent()	Saves the agent state. Called every save_frequency games.
set_to_eval_mode	[ ] Optional	set_to_eval_mode()	Switches to eval mode (e.g. epsilon -> 0, dropout off). Called in eval mode only.

Note: All optional methods are checked via hasattr before being called.

4. Execution Order per Game

```
__init__(player [, agent_path])
  |-- if agent_path is not None  -->  load_agent(agent_path)   # mandatory call in constructor

set_env_cfg(env_cfg)                                          # optional
[set_to_eval_mode()]                                          # optional, eval mode only

per step:
```

Interface: runner.py <-> Agent Class

```
  act(step, obs)
  update(step, obs, reward, action, terminated, next_obs)      # optional, training only

every N games:
  save_agent()                                                 # optional, training only
```

5. Example Pattern (as in agents/DQN/main.py)

```
def __init__(self, player: str, agent_path: Optional[str] = None):
    self.player = player
    self.name   = "My Agent"
    # ... network initialization ...
    self.q = QNet(self.state_dim, self.n_actions).to(self.device)
    if agent_path is not None:
        self.load_agent(agent_path)   # mandatory call

def load_agent(self, load_path: str):
    if os.path.isfile(load_path):
        self.q.load_state_dict(torch.load(load_path))
    else:
        print(f"Path {load_path} not found. Starting untrained.")

def save_agent(self):
    torch.save(self.q.state_dict(), self.agent_path)
```

Note: save_agent and load_agent are a matching pair: save_agent writes to self.agent_path, load_agent reads from the given path.